

Impact of Memory Hierarchy Parameters on Application Performance: A gem5 Simulation Study with Matrix Multiplication

Artur Fernandes Braga de Menezes
Computer Architecture
PUC Minas
Brazil

Artur Rizzi Martinho
Computer Architecture
PUC Minas
Brazil

Guilherme Meyer Guimaraes
Computer Architecture
PUC Minas
Brazil

Pedro Hosken Fernandes Guimaraes
Computer Architecture
PUC Minas
Brazil

Breno Moreira Cortez
Computer Architecture
PUC Minas
Brazil

Abstract—Cache hierarchy design is one of the most critical factors affecting processor performance for compute-intensive workloads. This paper presents a systematic simulation study using gem5 to evaluate the impact of four memory hierarchy parameters on the execution of a matrix multiplication benchmark: L1 data cache size, L2 cache size, main memory type, and L1 data cache associativity. Experiments were conducted in Syscall Emulation (SE) mode on an x86 single-core Timing CPU at 3 GHz with a classic two-level cache hierarchy. Results show that L2 capacity has the strongest performance impact – a transition from 32 KiB to 64 KiB reduces the L2 miss rate from 93.9% to 1.96% and yields a 17% speedup – while increasing L1 size beyond the working-set boundary reduces L1 miss rate dramatically but offers only 1.5% overall gain. L1D associativity above 4-way and main memory type variation produce negligible improvements under the studied conditions. These findings provide practical design insights for cache sizing in arithmetic-intensive applications.

Index Terms—gem5, cache hierarchy, memory simulation, matrix multiplication, cache miss, IPC, computer architecture

I. INTRODUCTION

The memory hierarchy remains one of the central design challenges in modern processor architecture. The gap between CPU clock frequencies and main memory access latencies – often referred to as the *memory wall* – forces architects to carefully tune multiple cache levels to mitigate performance degradation [1]. For compute-intensive workloads such as dense linear algebra, the spatial and temporal locality patterns of data accesses interact directly with cache capacity, block size, associativity, and replacement policy, making the choice of hierarchy parameters a decisive factor in achieving high instruction throughput [2].

Cycle-accurate simulation tools such as gem5 [3] allow researchers and students to explore this design space without

requiring physical hardware changes. gem5 provides a flexible, modular framework that models a wide range of memory hierarchies, CPU models, and ISAs, and has been widely used in architectural research to analyze the behavior of caches and memory subsystems [4].

This work presents a controlled simulation study using gem5 to answer the following questions:

- How does increasing L1 data cache size affect performance and cache miss behavior?
- What is the performance impact of varying L2 cache size?
 - Does upgrading the main memory technology (DDR3 vs. DDR4) yield measurable gains?
- How does L1 data cache associativity influence miss rates and overall execution time?

All experiments use the x86-matrix-multiply binary available in the gem5 standard resource library as the representative workload, executed in SE mode on a single-core x86 Timing CPU. This workload exhibits regular, stride-based memory access patterns characteristic of dense matrix operations, which stresses the data cache hierarchy in a predictable and analyzable way.

The main contributions of this paper are:

- 1) A reproducible, open-source experimental infrastructure built on top of gem5 v25.1.0.0 for evaluating cache hierarchy parameters;
- 2) Quantitative characterization of the performance sensitivity to L1 size, L2 size, memory technology, and

cache associativity under a matrix multiplication workload;

- 3) Identification of the working-set thresholds where cache parameters cease to deliver meaningful performance gains.

The remainder of the paper is organized as follows. Section II discusses related work. Section III describes the simulation environment and experimental design. Section IV presents and analyzes the results. Section V concludes and outlines future directions.

II. RELATED WORK

The gem5 simulator was introduced by Binkert et al. [3] as a merger between the M5 and GEMS simulators, providing a highly configurable full-system and SE-mode simulation platform. The tool has since been continuously updated; LowePower et al. [4] described the gem5 20.0 release, which introduced the *gem5 standard library* used in this work to compose memory hierarchies through high-level Python components.

Cache sizing and its relationship to workload performance has been extensively studied in the architecture literature. Cantin and Hill [5] performed a design-space exploration of cache configurations using SPEC CPU benchmarks and showed that the relative benefit of increasing cache capacity is strongly workload-dependent. Their findings align with this study’s observation that performance gains plateau once the working set fits within the cache.

Regarding matrix multiplication specifically, Lam et al. [6] demonstrated that cache-oblivious and cache-aware tiling strategies can dramatically reduce cache misses in dense linear algebra, and that even with a naïve triple-loop implementation, L1 miss behavior is governed by the matrix dimension relative to cache capacity. The x86-matrix-multiply benchmark used in this work follows a standard triple-loop structure, making it an appropriate proxy for cache stress evaluation.

The effect of cache associativity has been analyzed by Sugumar and Abraham [7], who showed that, for most workloads, increasing associativity beyond 4-way or 8-way yields diminishing returns because conflict misses represent a small fraction of total misses. Our results confirm this finding for the matrix multiplication workload studied.

More recently, simulation-based studies using gem5 for memory hierarchy exploration have been reported in several educational and research contexts [8]. These works validate gem5 as a reliable platform for architectural experiments and motivate its use in academic settings such as this course project.

III. METHODOLOGY

A. Simulation Environment

All simulations were executed on macOS using gem5 version v25.1.0.0 (commit 7a2b0e41), compiled with Clang 16.0.6 and Python 3.14.3 via SCons 4.10.1. The base configuration for all experiments is summarized in Table I.

The PrivateL1PrivateL2CacheHierarchy component from the gem5 standard library was used for all cache experiments, providing separate L1 instruction (L1I) and L1 data (L1D) caches backed by a unified L2 cache. The Python configuration scripts (configs/cache_exp.py, configs/assoc_exp.py, configs/memory_exp.py) parameterize cache sizes and memory types via command-line arguments, enabling automated batch execution.

TABLE I
BASE SIMULATION CONFIGURATION

Parameter	Value
Simulator	gem5 v25.1.0.0 (SE mode)
ISA	x86
CPU model	Timing (in-order)
Clock frequency	3 GHz
Number of cores	1
Cache line size	64 bytes
Memory	DDR3-1600, 1 GiB
Hierarchy	Private L1, Private L2 (classic)
Workload	x86-matrix-multiply

B. Workload

The benchmark x86-matrix-multiply is a standard matrix multiplication kernel distributed as part of the gem5 resource library. It performs a dense $N \times N$ matrix multiplication using a triple-loop (i, j, k) structure, which is known to exhibit high spatial locality in the inner loop for the two source matrices and lower temporal locality for the result matrix. All 26,728,189 instructions were simulated to completion in every run.

C. Experimental Design

Four independent experiments were conducted, each varying one parameter at a time:

Experiment A – L1 Cache Size: The L1D and L1I sizes were varied together across {16 KiB, 32 KiB, 64 KiB}, with L2 fixed at 256 KiB. The baseline is the 16 KiB configuration.

Experiment B – L2 Cache Size: The L2 size was varied across {32 KiB, 64 KiB, 128 KiB, 256 KiB}, with L1D and L1I fixed at 16 KiB. The baseline is the 32 KiB configuration.

Experiment C – Main Memory Type: Three DDR memory technologies were compared – DDR3-1600, DDR3-2133, and DDR4-2400 – using a fixed hierarchy of L1D/L1I = 16 KiB and L2 = 32 KiB (intentionally small to stress the main memory path). The baseline is DDR3-1600.

Experiment D – L1D Associativity: L1D associativity was varied across {2-way, 4-way, 8-way} with L1D/L1I size fixed at 16 KiB and L2 fixed at 64 KiB using the

PrivateL1SharedL2CacheHierarchy component (l1i_assoc = 8,

Notably, the L2 miss count remains nearly identical across all

	Speedup			L1D MPKI		
16 KiB	0.500	2.000	1.000	0.412%	2.38	1.45%
32 KiB	0.500	2.000	1.000	0.411%	2.37	1.48%
64 KiB	0.508	1.969	1.016	0.004%	0.02	23.6%

l2_assoc = 16). The baseline is 2way.

D. Metrics

The following metrics were extracted from the gem5 stats.txt output files:

- IPC (Instructions Per Cycle) and CPI (Cycles Per Instruction);
- Speedup relative to the baseline of each experiment;
- L1D / L1I / L2 hit and miss counts;
- Miss rate = misses / total accesses;
- MPKI (Misses Per Kilo-Instructions) = (misses / simInsts) × 1000;
- Average miss latency (in simulation ticks, where 1 tick = 1 ps at 1 THz simFreq).

IV. RESULTS AND ANALYSIS

A. Experiment A: L1 Cache Size

Table II presents the results for the L1 cache size variation. Moving from 16 KiB to 32 KiB produces a negligible improvement (speedup = 1.0001, Δ IPC < 0.001), with the L1D miss rate decreasing only marginally from 0.412% to 0.411%. This indicates that the L1D working set of the benchmark slightly exceeds 16 KiB but not 32 KiB in terms of conflict behavior.

The jump to 64 KiB, however, triggers a qualitative change: the L1D miss rate collapses from 0.411% to just 0.004%, reducing L1D MPKI from 2.37 to 0.021. This indicates that 64 KiB is sufficient to contain the entire active data working set, eliminating virtually all capacity and conflict misses at the L1 level. The corresponding speedup is 1.016 (1.6% faster) and CPI improves from 2.000 to 1.969.

MR = miss rate.

L1 Size	IPC	CPI	L1D MR	L2 MR
---------	-----	-----	--------	-------

The L2 miss rate anomaly at 64 KiB (rising to 23.6%) deserves attention: with L1D absorbing nearly all data requests, the few misses that escape to L2 are mostly cold misses that cannot be served there, resulting in a high L2 miss rate on a very small absolute miss count (995 misses). This is a statistical artifact of the reduced L2 access volume, not a sign of degraded L2 behavior.

B. Experiment B: L2 Cache Size

Table III shows the strong impact of L2 size. With L2 = 32 KiB, the L2 miss rate reaches 93.9%, meaning almost all L1D misses

Memory	IPC	CPI	Speedup	L2 Avg Miss Lat. (ticks)
DDR3-1600	0.427	2.342	1.000	52,512
DDR3-2133	0.430	2.324	1.008	49,926
DDR4-2400	0.428	2.336	1.003	51,627

three configurations ($\approx$ 993 L2 misses), while L2 hits drop sharply at 64 KiB because L1D misses that would have hit in L2 now hit in L1D directly. This confirms that the performance gain comes from avoiding the L1D→L2 penalty, not from reducing DRAM accesses.

TABLE II
EXPERIMENT A – L1 CACHE SIZE VARIATION (L2 = 256 KiB)

TABLE III
EXPERIMENT B – L2 CACHE SIZE VARIATION (L1D = L1I = 16 KiB)

L2 Size	IPC	CPI	Speedup	L2 Hits	L2 MR	L2 MPKI
32 KiB	0.427	2.342	1.000	4,192	93.9%	2.410
64 KiB	0.500	2.002	1.170	67,249	1.96%	0.050
128 KiB	0.500	2.000	1.171	67,593	1.46%	0.038
256 KiB	0.500	2.000	1.171	67,603	1.45%	0.037

unlocks most of the achievable performance improvement, while subsequent doublings bring negligible benefit.

C. Experiment C: Main Memory Technology

Table IV compares three DDR memory configurations. In this experiment, L2 is intentionally small (32 KiB) to produce a high rate of DRAM accesses ($\approx$ 64,400 L2 misses per run). Despite this, the performance differences across memory types are minimal: DDR3-2133 yields a 0.79% speedup over DDR31600, and DDR4-2400 yields only 0.28%.

TABLE IV
EXPERIMENT C – MAIN MEMORY TYPE (L1D = L1I = 16 KiB, L2 = 32 KiB)

escalate to main memory. This produces the worst IPC (0.427) and highest CPI (2.342) across all experiments.

Increasing L2 to 64 KiB yields a 17% speedup and reduces the L2 miss rate to 1.96%, because 64 KiB is sufficient to hold the matrix working set that repeatedly misses in the 16 KiB L1D. Beyond 64 KiB, further doubling (128 KiB and 256 KiB) yields only marginal additional gains (Δ speedup < 0.001), demonstrating a clear working-set saturation point at approximately 64 KiB for this benchmark.

These results highlight that L2 sizing is the dominant lever for this workload: a single capacity doubling (32→64 KiB) The limited sensitivity to memory type can be explained by two factors. First, even with a 32 KiB L2, the total number of DRAM accesses (64,400) represents a small fraction of the total

26.7M instructions. Second, the Timing CPU stalls the entire pipeline on each cache miss, so reducing DRAM latency improves execution time only proportionally to the fraction of cycles spent waiting for DRAM. With 53–62M total cycles and 64,400 DRAM requests each taking $\approx 50,000$ ticks (≈ 50 ns at 1 THz), the DRAM-bound fraction is non-trivial but not dominant enough for memory bandwidth differences to matter.

This result underscores a key architectural insight: upgrading memory technology alone is ineffective when the primary bottleneck is cache capacity – the correct intervention is to fix the cache hierarchy first.

D. Experiment D: L1D Cache Associativity

Table V shows the impact of L1D associativity. The 2-way configuration produces a dramatically higher L1D miss rate (2.649%) and MPKI (15.27) compared to 4-way (0.426% miss rate, MPKI 2.46) and 8-way (0.412%, MPKI 2.38). The 2-way to 4-way transition yields an 8.3% speedup, whereas the 4-way to 8-way transition yields only 0.05% additional gain.

The high miss rate at 2-way associativity signals a significant number of conflict misses: the matrix multiplication access pattern maps multiple matrix rows to the same cache

TABLE V

EXPERIMENT D – L1D ASSOCIATIVITY (L1D = 16 KiB, L2 = 64 KiB)

Assoc.	IPC	CPI	Speedup	L1D MR	L1D MPKI
2-way	0.461	2.169	1.000	2.649%	15.27
4-way	0.499	2.002	1.083	0.426%	2.46
8-way	0.500	2.001	1.084	0.412%	2.38

sets when using low associativity and a power-of-two matrix dimension, a well-known pathological case for directmapped and 2-way caches [6]. Increasing associativity to 4way resolves most conflicts, and 8-way provides only marginal additional improvement because capacity misses, not conflicts, dominate the remaining miss events.

The fact that 4-way delivers virtually the same performance as 8-way (speedup difference of 0.001) provides strong evidence that the optimal associativity point for this workload at 16 KiB is 4-way, offering a favorable area-power-performance tradeoff.

E. Summary of Findings

Table VI summarizes the relative speedup gains across all experiments. The dominant takeaway is that *L2 capacity* is the most impactful parameter for this workload, followed by *L1D associativity* (when it is too low), *L1D capacity* (only above a working-set threshold), and finally *main memory technology* (negligible when cache sizing is inadequate).

TABLE VI

MAXIMUM SPEEDUP PER EXPERIMENT (RELATIVE TO EACH BASELINE)

Experiment	Max Speedup
------------	-------------

B: L2 32→64 KiB	1.170
D: L1D assoc 2→4-way	1.083
A: L1D 16→64 KiB	1.016
C: Memory DDR3-1600→2133	1.008

V. CONCLUSIONS

This paper presented a simulation-based study of memory hierarchy parameters for a matrix multiplication workload using gem5 v25.1.0.0 in SE mode on an x86 Timing CPU. Four dimensions were evaluated: L1 cache size, L2 cache size, main memory technology, and L1D cache associativity. The key findings are:

- 1) L2 size is the dominant performance lever. A transition from 32 KiB to 64 KiB reduced the L2 miss rate from 93.9% to 1.96% and yielded a 17% speedup. Beyond this threshold, gains plateau because the working set is fully captured in L2.
- 2) L1D size matters only near working-set boundaries. Doubling L1D from 16 to 32 KiB had no measurable effect; doubling to 64 KiB reduced L1D MPKI from 2.38 to 0.02, yielding a 1.6% speedup. This underscores the importance of matching cache capacity to the actual working set, not simply doubling capacity.
- 3) Memory technology contributes marginally when cache capacity is inadequate. With a 32 KiB L2 forcing DRAM traffic, DDR3-2133 offered only 0.79% improvement over DDR3-1600, and DDR4-2400 only 0.28%. The correct first step is to fix the cache hierarchy.
- 4) L1D associativity is critical at low values. 2-way associativity caused severe conflict-miss flooding (MPKI = 15.27) for this access pattern. Moving to 4-way resolved most conflicts and delivered an 8.3% speedup. 8-way provided no additional benefit, suggesting that 4way is the Pareto-optimal point for this workload.

Future work directions include: (i) extending the study to multi-core configurations to evaluate the impact of shared vs. private L2 on coherence traffic; (ii) varying cache line size to analyze its interaction with stride-based access patterns; (iii) employing out-of-order CPU models (O3) to assess whether memory-level parallelism changes the sensitivity to cache parameters; and (iv) using cache-optimized workloads (e.g., tiled matrix multiplication) to compare miss behavior against the naïve triple-loop variant.

ACKNOWLEDGMENT

The authors thank the gem5 community for maintaining an open-source simulation infrastructure and the course instructors for the project guidance.

REFERENCES

- [1] W. A. Wulf and S. A. McKee, “Hitting the memory wall: Implications of the obvious,” *ACM SIGARCH Computer Architecture News*, vol. 23, no. 1, pp. 20–24, Mar. 1995.

- [2] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2019.
- [3] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, pp. 1–7, Aug. 2011.
- [4] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, A. Armejach, N. Asmussen, S. Bharadwaj, G. Black *et al.*, "The gem5 simulator: Version 20.0+," *arXiv preprint arXiv:2007.03152*, 2020.
- [5] J. F. Cantin and M. D. Hill, "Cache performance for selected SPEC CPU2000 benchmarks," *ACM SIGARCH Computer Architecture News*, vol. 29, no. 4, pp. 13–18, Sep. 2001.
- [6] M. S. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Santa Clara, CA, Apr. 1991, pp. 63–74.
- [7] R. A. Sugumar and S. G. Abraham, "Efficient simulation of caches under optimal replacement with applications to miss characterization," in *Proc. 1993 ACM SIGMETRICS Conf. Measurement and Modeling of Computer Systems*, Santa Clara, CA, May 1993, pp. 24–35.
- [8] J. Akram and L. Sawalha, "A survey of computer architecture simulation techniques and tools," *IEEE Access*, vol. 7, pp. 9800–9828, 2019, doi: 10.1109/ACCESS.2019.2930584.

Link for the repositor Where the experiments were conducted:
<https://github.com/guimeyer2/gem5-memory-hierarchy-study>